

PROJET D'ÉTUDES — RENDU INDIVIDUEL

ToolboxV8

Plateforme d'automatisation de tests d'intrusion

Auteur	Abdallah NASR
Rôle principal	Analyste / Intégration des outils offensifs / Assurance qualité — 5 phases du pentest, profils, tests end-to-end, KPIs
Promotion	Mastère Cybersécurité 2025/2026
École	SUP DE VINCI
Date	Juin 2026

1. Perspectives d'évolution

1.1 Automatiser les tests d'intégration des outils

Aujourd'hui, j'ai validé chaque outil à la main sur des cibles réelles. La vraie amélioration serait une **suite de tests automatisés** : un jeu de cibles de référence (conteneurs vulnérables) et des assertions sur les sorties attendues de chaque module. On détecterait ainsi les régressions au moindre changement, au lieu de les découvrir en démo. C'est exactement ce qui nous a coûté du temps sur ZAP et SSLyze.

1.2 Automatiser complètement Metasploit

Le module Metasploit est codé et invocable via `pymetasploit3`, mais il nécessite le lancement manuel du démon `msfrpcd`. Un sidecar `msfrpcd` préconfiguré dans le `docker-compose` rendrait le seul outil offensif encore non démontrable end-to-end pleinement opérationnel depuis l'interface.

1.3 Enrichir les dictionnaires et wordlists

L'intégration de SecLists (plus de 1000 wordlists prêtes à l'emploi) apporterait une vraie profondeur aux modules Hydra, John et Gobuster, sans alourdir l'expérience analyste grâce au système de profils déjà en place.

1.4 Rendre la post-exploitation opérationnelle

Le module `post_exploit` est aujourd'hui documentaire (il propose des commandes d'énumération sans les exécuter). Le faire passer à une exécution réelle, encadrée et journalisée, compléterait la couverture des cinq phases du pentest de façon concrète.

1.5 Affiner les profils selon les retours analystes

Les profils `Quick` / `Standard` / `Full` sont un bon point de départ, mais ils gagneraient à être calibrés sur la base de retours terrain réels (équilibre vitesse/exhaustivité par type de cible), pour coller au plus près de l'usage d'un analyste.

2. Analyse critique — limites et retours d'expérience

2.1 ZAP « terminé en 2 secondes » — le piège du fire-and-forget

Mon intégration initiale de ZAP lançait le scan actif et rendait la main immédiatement, sans attendre la fin ni récupérer les alertes : le rapport affichait un scan « terminé » mais vide. J'ai refondu l'intégration avec un **polling** jusqu'à 100 % et la récupération des alertes agrégées par sévérité (jusqu'à 105 alertes remontées sur une cible). **Leçon** : une intégration ne vaut que si elle produit un résultat *exploitable*, pas seulement un code de retour.

2.2 SSLyze qui plantait en silence dans le conteneur

SSLyze échouait avec un `Network is unreachable` lié à une tentative de connexion IPv6 dans un réseau Docker sans route v6. J'ai réimplémenté la résolution avec un **pré-check TCP** et un fallback **IPv4 d'abord, IPv6 ensuite**, plus des messages d'erreur clairs. **Leçon** : les environnements conteneurisés ont leurs propres pièges réseau, qu'il faut anticiper.

2.3 SQLmap dump, une limite intrinsèque assumée

Sur cible distante, le dump SQLmap pouvait durer des heures à cause de l'injection blind imposée par le WAF. J'ai dû **brider le profil** (nombre de lignes limité, technique boolean-based, threads contrôlés) pour rester dans un temps raisonnable. Même bridé, l'extraction ne réussit pas toujours. C'est une limite intrinsèque de l'injection aveugle que nous avons choisi de **documenter honnêtement** plutôt que de masquer.

2.4 La validation de cible dupliquée

La logique de validation des cibles était présente à la fois côté front et côté back. Quand il a fallu l'assouplir pour `passive_recon` (cible en champ libre) et Hydra (hostname Docker), j'ai dû intervenir à plusieurs endroits. Une validation centralisée aurait évité ces doublons et les risques d'incohérence.

2.5 Trop de validation manuelle

La majorité de mes tests d'intégration ont été faits à la main. C'est ce qui m'a permis de repérer les bugs (ZAP, SSLyze), mais tardivement. Des tests automatisés dès le départ auraient capté ces problèmes bien plus tôt.

3. Analyse personnelle

3.1 Ce que ce projet m'a vraiment appris

Mon rôle d'analyste m'a fait comprendre que **intégrer un outil est facile ; garantir qu'il produit un résultat juste et exploitable est le vrai travail**. À chaque module, je me suis posé la question : « est-ce que ce qui remonte dans le rapport est fiable ? ». Cette exigence de fiabilité est, je pense, le cœur du métier d'analyste en sécurité : un audit n'a de valeur que si on peut faire confiance à ses résultats.

J'ai aussi développé une vraie **culture des cinq phases du pentest** : en intégrant les outils de la reconnaissance à l'exploitation, j'ai compris comment ils s'enchaînent et se complètent, ce qui dépasse la simple connaissance d'un outil isolé.

3.2 L'utilisation de l'IA dans le projet

J'ai utilisé l'IA comme **assistant** pour accélérer l'intégration des outils : génération de squelettes de code pour parser les sorties, aide au débogage quand un outil se comportait de façon inattendue, et recherche d'options en ligne de commande. Mon constat rejoint celui de l'équipe : l'IA fait gagner du temps **sur ce qu'on comprend déjà**, mais elle ne remplace pas la vérification. Plusieurs fois, elle a proposé du code plausible qui ne tenait pas compte des spécificités de notre environnement conteneurisé (le piège réseau de SSLyze, par exemple). Je l'ai donc utilisée comme un accélérateur à valider, jamais comme une autorité.

3.3 Forces

- **Regard critique d'analyste** : ne jamais faire confiance à une sortie d'outil sans la vérifier, savoir distinguer un vrai résultat d'un faux positif.
- **Ténacité d'intégration** : refonte de ZAP et de SSLyze jusqu'à un résultat réellement exploitable.
- **Sens du cadre éthique** : tests menés exclusivement sur des cibles autorisées, jamais sur des systèmes tiers.
- **Esprit de démonstration** : création de cibles de test reproductibles (`pentest_target` , hashes de démo) pour des démos fiables et rejouables.

3.4 Faiblesses

- **Trop de tests manuels** : j'ai découvert certains bugs tard faute d'automatisation.
- **Standardisation tardive** des formats de sortie, ce qui a généré des allers-retours avec le reporting.
- **Tendance à approfondir un outil** avant d'avoir une couverture large fonctionnelle.

3.5 Compétences développées

- **Sécurité offensive** : maîtrise de Nmap, SQLmap, Hydra, John, ZAP, Gobuster, Metasploit et de la méthodologie des cinq phases.

- **Cryptologie appliquée** : cassage de mots de passe en ligne (Hydra) et hors ligne (John), compréhension des formats de hash.
- **Analyse de vulnérabilités** : interprétation d'injections SQL, d'alertes ZAP, d'audits TLS, de chemins cachés.
- **OSINT** : conception d'un catalogue de Google Dorks pour le renseignement passif.
- **Assurance qualité** : campagne de tests end-to-end, mesure de KPIs, validation sur cibles autorisées.

3.6 Pour les prochains projets

- **Écrire des tests d'intégration automatisés dès le début**, même simples.
- **Standardiser tôt** le format de sortie commun à tous les modules.
- **Viser d'abord une couverture large** fonctionnelle, puis approfondir chaque outil.
- **Documenter les limites honnêtement** (comme pour SQLmap) : c'est une marque de maturité professionnelle, pas un aveu de faiblesse.

Document rédigé dans le cadre du Projet d'Études — Mastère Cybersécurité 2025/2026 — SUP DE VINCI